

Chat multiusuario en red local

Diseño e implementación de un protocolo cliente-servidor con
programación concurrente, hilos de ejecución y sockets

Estudiante del curso *Modelado y Programación*
(Rocha Haro Cesar Antonio)

28 de septiembre de 2026

Índice

1	Resumen	2
2	Introducción	2
3	Análisis y diseño	3
3.1	El protocolo como punto de partida	4
3.2	Decisiones de arquitectura y alternativas consideradas	5
3.3	Primera iteración: la versión en C	6
3.4	Segunda iteración: diseño final en C#	8
3.5	Trazabilidad protocolo – código	11
4	Implementación	12
4.1	Tecnologías utilizadas	12
4.2	Requisitos para ejecutar el proyecto	12
4.3	Instrucciones de compilación y ejecución	12
4.4	Estado de las pruebas	12
5	Conclusiones	13
6	Bibliografía	14

1. Resumen

Este proyecto resuelve el problema de conectar a múltiples usuarios dentro de una red local para que puedan comunicarse mediante mensajes de texto, siguiendo al pie de la letra un protocolo de aplicación basado en JSON proporcionado por el profesor del curso. Para lograrlo se construyó un sistema cliente-servidor: un servidor que acepta conexiones TCP y atiende a cada cliente en un hilo de ejecución independiente, sincronizando el acceso al estado compartido (usuarios, salas y sus membresías) mediante exclusión mutua; y un cliente de consola que se conecta al servidor, envía comandos del protocolo y muestra en tiempo real lo que otros usuarios van publicando. Ambos programas se diseñaron siguiendo el patrón Modelo-Vista-Controlador. El proyecto tuvo dos iteraciones: una primera versión del servidor escrita en C, que sirvió para entender a fondo el problema pero que se abandonó por las dificultades de simular orientación a objetos, manejar memoria manualmente y sincronizar hilos con mutex de bajo nivel; y una segunda versión, la que finalmente se entrega, escrita íntegramente en C# sobre .NET, usando `System.Text.Json` para el formato del protocolo. El resultado es un chat funcional que soporta identificación de usuarios, cambios de estado, mensajes públicos y privados, y salas privadas con invitación, todo de acuerdo con lo especificado en el protocolo.

2. Introducción

El problema central que resuelve este proyecto es, en su forma más simple, **¿cómo conectar a múltiples personas dentro de una red local para que puedan intercambiar mensajes de texto entre sí?** Al tratarse de una red local (y no de Internet abierto), el problema encaja de inmediato en un problema clásico de redes de cómputo resuelto con *sockets*: un programa servidor escucha en un puerto conocido y los clientes se conectan a él.

En el análisis inicial del problema surgieron, en orden, las siguientes preguntas guía —numeradas aquí porque se retoman más adelante en la sección de Análisis y Diseño—:

1. ¿Cómo conectamos a múltiples personas en una red?
2. Dado que habrá múltiples usuarios conectados, ¿cómo se va a lidiar con todos ellos al mismo tiempo?
3. ¿La comunicación será directamente entre los clientes?
4. ¿O habrá un intermediario (el servidor) que reenvíe los mensajes a los usuarios correspondientes?
5. Si hay múltiples usuarios modificando el mismo estado compartido, ¿qué sucede si dos de ellos piden acceder al mismo lugar en memoria al mismo tiempo?
6. ¿A cuál de los dos se le da prioridad?
7. ¿Qué pasa si un usuario elimina cierta información (por ejemplo, al desconectarse) justo cuando otro usuario la está solicitando?
8. ¿Cómo se entera ese segundo usuario de que la información cambió?
9. ¿Qué pasa si hay tantos usuarios conectados que ya no alcanza la memoria del servidor para admitir más?
10. ¿Qué paradigma de programación se acomoda mejor a este problema?

11. ¿El problema es siquiera computable?
12. Si lo es, ¿se puede resolver de manera eficiente, o hace falta recurrir a herramientas distintas de los algoritmos de tiempo polinomial?

Las preguntas (1) a (4) apuntan directamente a la necesidad de una arquitectura cliente-servidor con concurrencia; las preguntas (5) a (9) anticipan, correctamente, que el verdadero reto del proyecto no es abrir un socket sino **sincronizar el acceso a estado compartido entre varios hilos de ejecución**; y las preguntas (10) a (12) son, en retrospectiva, más una reflexión académica que un obstáculo real: el problema sí es computable, sí es eficientemente resoluble (basta con estructuras de datos en memoria como diccionarios/tablas de dispersión y una técnica de sincronización razonable), y el paradigma imperativo/orientado a objetos con soporte de hilos nativo —como el que ofrece C#— resultó ser una opción cómoda para expresarlo.

Objetivo

Construir un servidor y un cliente que implementen *exactamente* el protocolo de mensajería proporcionado por el profesor (identificación, cambio de estado, lista de usuarios, texto privado y público, salas privadas con invitación, y desconexión limpia), usando programación concurrente, hilos de ejecución y sockets, y organizando el código de ambos programas con el patrón Modelo-Vista-Controlador.

Alcance

El proyecto cubre comunicación dentro de una misma red local, con las operaciones exactas que define el protocolo. Explícitamente **no** se contemplan: comunicación con usuarios fuera de la red local; ningún mecanismo de autenticación o contraseña (cualquiera puede identificarse con cualquier nombre de usuario disponible); cifrado del tráfico (los mensajes viajan en texto plano); protección contra usuarios maliciosos que ataquen al servidor; ni ocultamiento de las direcciones IP de los usuarios, que quedan expuestas a quien tenga acceso al código o al proceso del servidor. El diseño es razonable para un rango aproximado de cientos hasta unos 10,000 usuarios concurrentes; servir a una escala de millones de usuarios requeriría cambios de arquitectura (por ejemplo, múltiples instancias de servidor y un mecanismo de mensajería entre ellas) que quedan fuera del alcance de este proyecto.

Facilidad de partida y trabajo futuro

Contar con un protocolo ya definido por el profesor —con el formato exacto de cada mensaje JSON, sus respuestas y sus casos de error— eliminó por completo la ambigüedad sobre *qué* debía construirse, dejando como único problema de ingeniería el *cómo*. Como trabajo futuro, es totalmente viable integrar autenticación de usuarios, cifrado de la conexión (por ejemplo con TLS), mejores prácticas de manejo de direcciones IP, y optimizaciones para escalar a un número mucho mayor de usuarios; estos puntos se retoman con más detalle en la sección de Conclusiones.

3. Análisis y diseño

Esta sección explica cómo el protocolo dado guió cada decisión de diseño, qué alternativas se consideraron, y cómo evolucionó la arquitectura entre la primera iteración del proyecto (en C, incompleta) y la segunda (en C#, la que se entrega).

3.1 El protocolo como punto de partida

El protocolo especifica doce mensajes que el servidor puede recibir y sus respuestas/efectos correspondientes. La tabla 1 resume esa relación tal como se definió en el documento del protocolo, y es, en buena medida, la tabla de la que se derivó directamente el diseño de la clase **Servidor** (un método por operación) y del controlador que enruta cada mensaje entrante hacia ese método.

Mensaje	Quién lo envía	Efecto principal	A quién se notifica
IDENTIFY	Cliente	Registra el nombre de usuario o responde <code>USER_ALREADY_EXISTS</code>	Todos (<code>NEW_USER</code>)
STATUS	Cliente	Cambia el estado (<code>ACTIVE/AWAY/BUSY</code>) si es distinto al actual	Todos (<code>NEW_STATUS</code>)
USERS	Cliente	Consulta la lista de usuarios y sus estados	Solo quien preguntó (<code>USER_LIST</code>)
TEXT	Cliente	Envía texto privado a un usuario existente	Destinatario (<code>TEXT_FROM</code>)
PUBLIC_TEXT	Cliente	Envía texto a todos los conectados	Todos menos el emisor (<code>PUBLIC_TEXT_FROM</code>)
NEW_ROOM	Cliente	Crea una sala nueva; el creador queda dentro	Solo el creador (<code>RESPONSE</code>)
INVITE	Cliente en la sala	Invita a uno o más usuarios existentes	Cada invitado (<code>INVITATION</code>)
JOIN_ROOM	Cliente invitado	Une al usuario si fue invitado previamente	Sala (<code>JOINED_ROOM</code>)
ROOM_USERS	Cliente en la sala	Consulta usuarios y estados de la sala	Solo quien preguntó (<code>ROOM_USER_LIST</code>)
ROOM_TEXT	Cliente en la sala	Envía texto a la sala	Sala, menos el emisor (<code>ROOM_TEXT_FROM</code>)
LEAVE_ROOM	Cliente en la sala	Saca al usuario de la sala; si queda vacía, la sala desaparece	Sala (<code>LEFT_ROOM</code>)
DISCONNECT	Cliente	Cierra la conexión y abandona todas sus salas	Todos (<code>DISCONNECTED</code>) y cada sala (<code>LEFT_ROOM</code>)

Cuadro 1: Mensajes del protocolo y su efecto, tal como los define el documento proporcionado por el profesor. Esta tabla es, en esencia, la especificación de la que se derivó la interfaz pública de la clase **Servidor**.

Otros detalles del protocolo que influyeron directamente en el diseño:

- Los mensajes viajan como una línea de JSON terminada en `"\n"` sobre un flujo TCP, y TCP no garantiza que una sola lectura del socket corresponda a un solo mensaje. Esto obligó a diseñar una clase dedicada exclusivamente a *acumular bytes y partirlos en líneas* (la clase **BufferLineas**, discutida más adelante), en vez de asumir ingenuamente que `Read()` devuelve un mensaje completo.

- Un cliente que manda cualquier mensaje antes de `IDENTIFY` debe recibir `NOT_IDENTIFIED` y ser desconectado; un mensaje mal formado debe recibir `INVALID` y también desconectar. Esto define un pequeño autómata de dos estados (*no identificado* / *identificado*) que el controlador de cada conexión debe respetar antes de despachar cualquier operación.
- Los límites de longitud (8 caracteres para nombres de usuario, 16 para nombres de sala) y las reglas de limpieza (una sala desaparece cuando se queda sin usuarios) son responsabilidades de *negocio*, no de red, por lo que se ubicaron en el Modelo y no en el Controlador.

3.2 Decisiones de arquitectura y alternativas consideradas

Cliente-servidor con un hilo por conexión. El protocolo es inherentemente *stateful* por conexión (un cliente debe identificarse antes de hacer cualquier otra cosa, y el servidor debe recordar su nombre, estado y salas mientras la conexión viva). La forma más directa de modelar eso es que cada conexión aceptada tenga su propio hilo que hace lecturas bloqueantes del socket en un ciclo; ese hilo es, efectivamente, el lugar natural donde vive el estado de "en qué parte del protocolo va este cliente". Se consideró la alternativa de E/S asíncrona sin hilos dedicados (`async/await` sobre `NetworkStream`, o `SocketAsyncEventArgs`), que escala mejor cuando hay miles de conexiones mayormente inactivas; se descartó para este proyecto porque (a) el enunciado pide explícitamente el uso de *hilos de ejecución*, y (b) para el rango de usuarios contemplado en el alcance, un hilo por conexión es perfectamente viable y bastante más fácil de razonar correctamente.

Un único candado global en el servidor. El estado compartido del servidor —la tabla de usuarios y la tabla de salas— se protege con un solo objeto de bloqueo (`lock`) dentro de la clase `Servidor`. Se consideró usar un candado por colección, o incluso un candado por sala, para permitir más paralelismo entre operaciones que no se estorban entre sí (por ejemplo, dos usuarios mandando texto a dos salas distintas). Se descartó esa alternativa por dos razones: primero, varias operaciones del protocolo (en particular `INVITE`, que debe revisar la tabla de salas *y* la de usuarios de forma atómica para no invitar a alguien que se desconectó a medio camino) necesitan ver ambas tablas de forma consistente, lo que con candados separados exigiría un orden de adquisición cuidadosamente diseñado para evitar *deadlocks*; segundo, el volumen de mensajes de un chat —limitado por la velocidad de tecleo humana— está muy lejos de justificar esa complejidad adicional. Un solo candado sacrifica algo de paralelismo teórico a cambio de una implementación mucho más fácil de verificar como correcta.

MVC en ambos programas. Se siguió el patrón Modelo-Vista-Controlador tanto en el servidor como en el cliente, tal como lo recomendó el profesor en clase. La razón práctica es doble: primero, separa con claridad *qué* hace cada mensaje del protocolo (Modelo) de *cómo* llega ese mensaje por el socket (Controlador), lo cual —como se detalla en la sección 3.3— fue precisamente el error de diseño que arrastró la primera versión en C; segundo, permite sustituir la vista de consola del cliente por una interfaz gráfica en el futuro sin tocar una sola línea del Modelo o del Controlador.

`System.Text.Json`. Se evaluaron dos formas de manejar JSON en C#: bibliotecas de terceros (como `Newtonsoft.Json`) y la biblioteca incluida en el propio runtime de .NET, `System.Text.Json`. Se optó por la segunda porque no agrega ninguna dependencia externa al proyecto y porque ofrece las dos formas de acceso que el proyecto necesita: serialización fuertemente tipada a partir de objetos anónimos de C# para *construir* cada mensaje saliente (el "shape" del JSON queda definido por la sintaxis del objeto anónimo), y un modelo de objetos dinámico (`JsonNode/JsonObject`) para *leer*

mensajes entrantes cuya forma exacta depende del campo `"type"`. Se consideró también definir una clase (DTO) por cada uno de los doce tipos de mensaje, con deserialización fuertemente tipada de extremo a extremo; se descartó por ahora porque, con solo doce formas de mensaje, el costo de mantener doce clases adicionales no se justificaba frente a leer los campos dinámicamente, aunque se reconoce como un refactor natural si el protocolo creciera.

3.3 Primera iteración: la versión en C

Antes de la versión que se entrega, el proyecto se intentó construir por completo en `C` para el servidor. La motivación fue puramente académica: el profesor mencionó que programar el servidor en `C` daba un punto extra, así que, aunque la primera inclinación del autor era usar un lenguaje orientado a objetos, se decidió intentarlo en `C` primero.

El plan de trabajo para llegar a un proyecto funcional fue el siguiente:

1. Entender a fondo el protocolo proporcionado.
2. Construir un esqueleto de servidor: aceptar un cliente y hacerle *echo* de sus mensajes.
3. Construir un esqueleto de cliente: conectarse al servidor, mandar mensajes y leer el eco, para confirmar que cliente y servidor podían comunicarse.
4. Agregar soporte para varios clientes simultáneos: un hilo por cliente y una lista de clientes compartida protegida con un mutex.
5. Refactorizar todo hacia MVC una vez que las piezas fundamentales funcionaran.
6. Pulir: corregir bugs y, si el tiempo alcanzaba, intentar una interfaz gráfica.
7. Escribir la documentación.

Bajo ese plan se llegó a diseñar la estructura de carpetas `modelo/vista/controlador` y las siguientes clases, simuladas en `C` mediante *structs* y archivos de cabecera (`.h`) que declaraban las funciones asociadas a cada "clase":

- **Modelo.** `Servidor`, compuesto por sus salas y sus usuarios; `Sala`, con sus usuarios y los sockets/puertos conectados a ella; y `Usuario`, con su nombre, su socket, un arreglo de las salas a las que se había unido, las invitaciones recibidas y las personas que él mismo había invitado. Este último diseño —el de `Usuario`— se consideró acertado desde el principio y se conservó conceptualmente en la segunda iteración.
- **Controlador.** `conexionHandler`, encargado de toda la gestión de hilos *y*, además, de cada acción a nivel de JSON que debía realizar el servidor; y `mensajeDispatcher`, encargado de leer línea por línea lo que el servidor iba recibiendo del socket.
- **Vista.** Nunca se llegó a completar ninguna clase de esta capa: el desarrollo se quedó atorado en el Modelo y el Controlador.

La figura 1 muestra ese diseño tal como se concibió, señalando en rojo los dos problemas de diseño que el propio desarrollo del proyecto dejó en evidencia y que se corrigieron en la segunda iteración:

1. La idea de que `Sala` funcionara como una especie de *mini-servidor*, con responsabilidades parecidas a las de `Servidor` pero acotadas a su propia membresía. En la práctica esto habría duplicado el problema de sincronización de hilos (habría que sincronizar el servidor *y*, por separado, cada sala), en vez de resolverlo una sola vez.

2. Que `conexionHandler` concentrara tanto la gestión de hilos (una responsabilidad de Controlador) como la lógica de qué hacer con cada mensaje del protocolo (una responsabilidad de Modelo). Mezclar ambas cosas fue, en palabras del propio desarrollo, la causa de que incluso implementar IDENTIFY —la operación más simple del protocolo— resultara mucho más difícil de lo que debía ser.

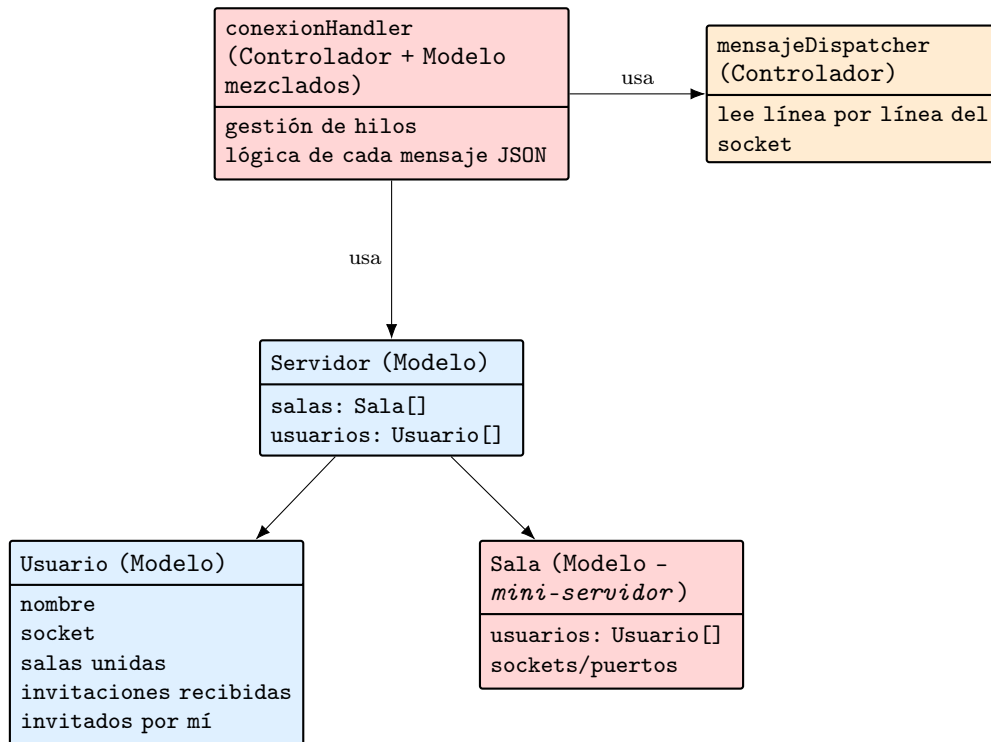


Figura 1: Diseño original de la primera iteración (en C). En rojo, los dos puntos que el propio proceso de desarrollo identificó como errores de diseño: `Sala` concebida como un mini-servidor redundante, y `conexionHandler` mezclando responsabilidades de Controlador y de Modelo.

Más allá del diseño, las dificultades *de implementación* en C fueron determinantes para abandonar esta versión:

- **Simular objetos.** C no tiene encapsulación real; las "clases" se simulaban con *structs* y referencias en memoria, y cada "clase" debía diseñarse por completo en su archivo de cabecera antes de poder escribir una sola línea de su implementación, lo que hacía el ciclo de desarrollo mucho más lento que en un lenguaje orientado a objetos.
- **JSON en C.** Existen bibliotecas de parseo de JSON para C, pero ninguna resuelve el problema por completo: sin importar cuánto trabajo ahorre la biblioteca, seguía siendo necesario escribir a mano una cantidad considerable de código para cumplir el protocolo al pie de la letra.
- **Hilos y memoria compartida.** Evitar que dos hilos alteraran el mismo lugar en memoria al mismo tiempo se resolvió con mutex de `pthread`s, pero administrar manualmente cuándo bloquear y desbloquear cada recurso resultó, con mucho, la parte más difícil de todo el proyecto.
- **Pruebas.** Contrario a lo anterior, escribir pruebas unitarias fue relativamente sencillo gracias a la biblioteca `cmocka`, que funcionó bien para este propósito.

La combinación de manejo manual de memoria, bibliotecas de JSON incompletas y sincronización de hilos de bajo nivel hizo que el desarrollo avanzara muy lento; ante la falta de tiempo, se tomó la decisión de descartar por completo la versión en C y reiniciar el proyecto en C#, decisión que se retoma con más detalle en las Conclusiones.

3.4 Segunda iteración: diseño final en C#

La versión que finalmente se entrega aplica directamente las lecciones de la primera iteración. Se organizó en **tres proyectos** de .NET (figura 2): **server** y **client**, cada uno con su propio Modelo, Vista y Controlador; y **chatComun**, una biblioteca compartida que contiene únicamente el código de bajo nivel que *ambos* programas necesitan de forma idéntica —partir el flujo de bytes en líneas y serializar/leer JSON—, de modo que esa lógica no se duplique en dos lugares distintos como habría ocurrido de seguir separando el proyecto en solo dos partes.

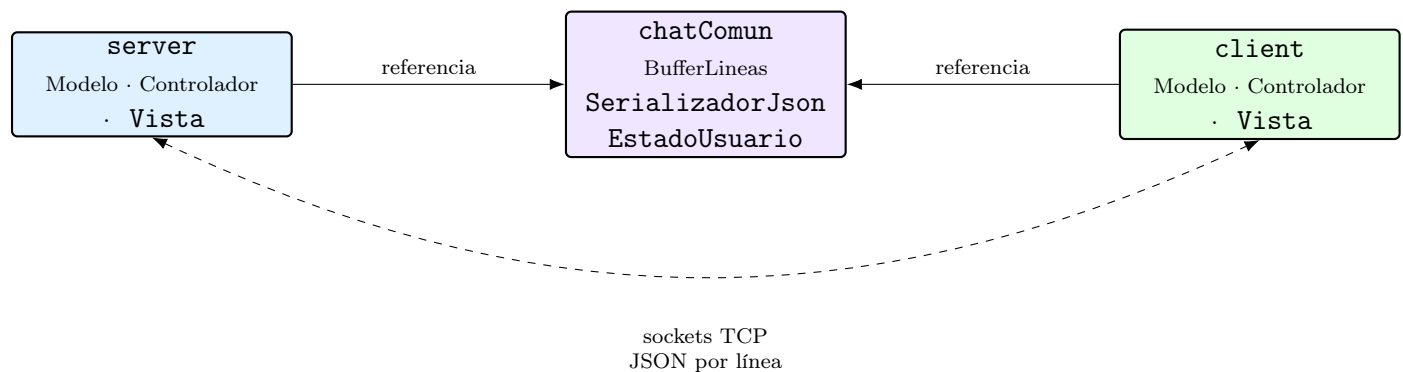


Figura 2: Arquitectura general: dos programas independientes que comparten únicamente el código de bajo nivel del protocolo a través de **chatComun**.

Diagrama de clases del servidor

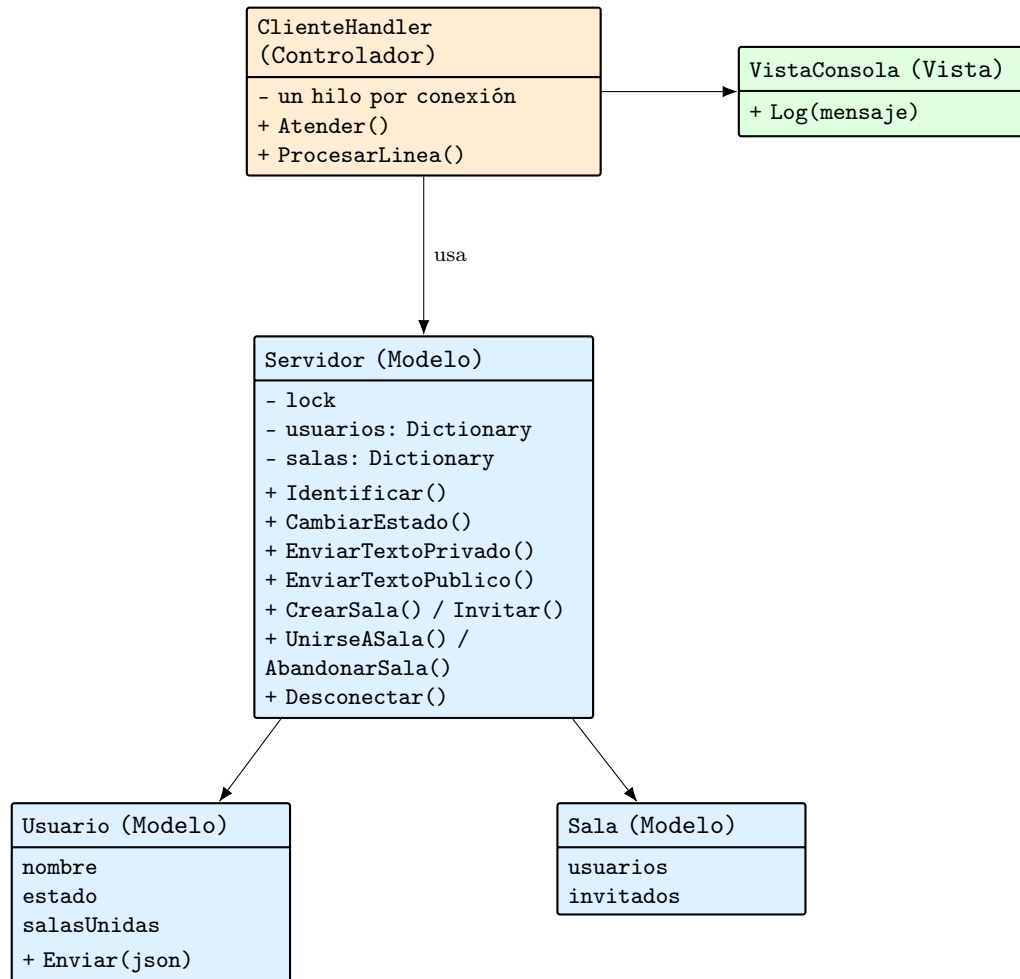


Figura 3: Servidor en C#: **Servidor** concentra toda la lógica de negocio y la sincronización con un único candado; **ClienteHandler** es un controlador delgado, uno por conexión, que solo traduce JSON en llamadas al Modelo. **Usuario** y **Sala** ya no intentan actuar como servidores por su cuenta: toda la membresía se resuelve de forma centralizada, corrigiendo el problema señalado en la figura 1.

Diagrama de clases del cliente

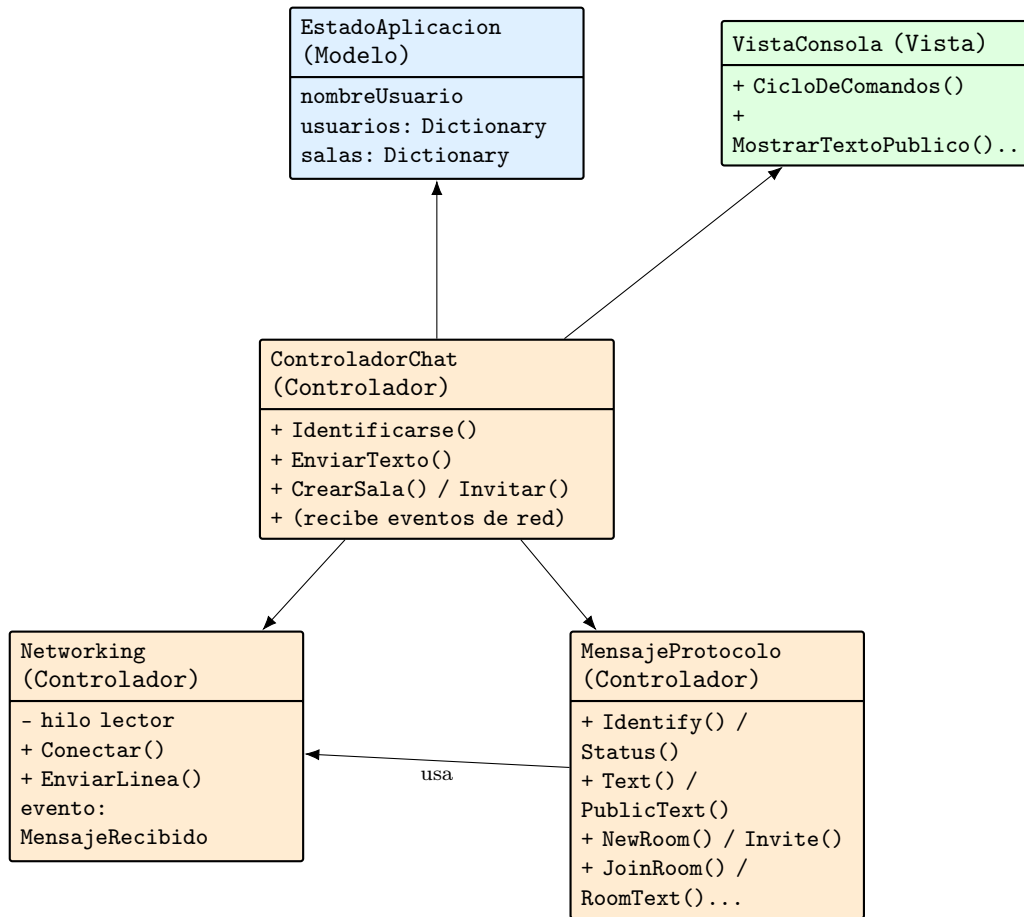


Figura 4: Cliente en C#: **Networking** es la clase dedicada exclusivamente a la conexión TCP; **MensajeProtocolo** construye cada mensaje del protocolo y lo envía a través de **Networking**; **ControladorChat** conecta ambas con el Modelo (**EstadoAplicacion**) y la Vista.

Ejemplo de flujo: IDENTIFY

La figura 5 ilustra por qué el hilo por conexión y el candado del servidor son necesarios incluso para la operación más simple del protocolo: identificar a un usuario debe, de forma atómica, revisar que el nombre no exista y registrarlo, y además debe notificarle a *todas los demás* clientes conectados —cada uno atendido por su propio hilo— sin que dos identificaciones simultáneas dejen la tabla de usuarios en un estado inconsistente.

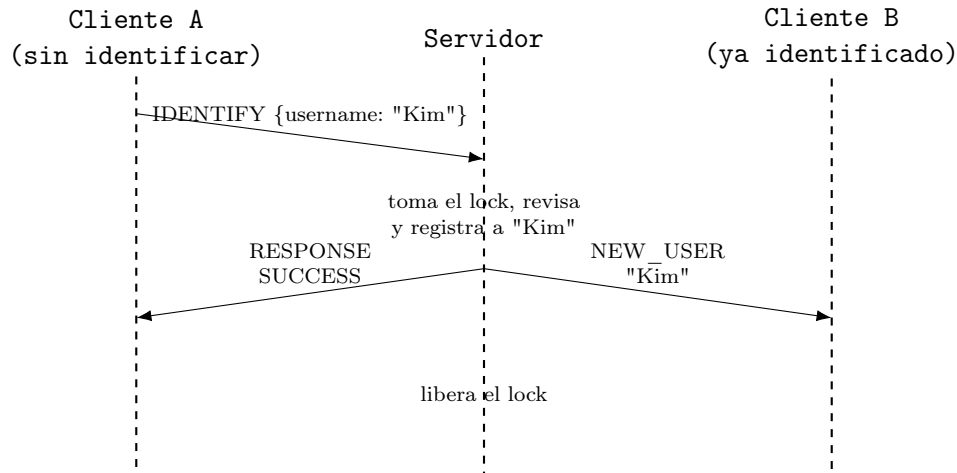


Figura 5: Al identificarse, el servidor responde solo a quien lo pidió, pero además difunde **NEW_USER** a todos los demás clientes ya conectados, cada uno en su propio hilo; el candado global garantiza que ninguna otra identificación concurrente vea la tabla de usuarios a medio actualizar.

3.5 Trazabilidad protocolo – código

La tabla 2 muestra, para cada mensaje que el servidor recibe, el método del Modelo que implementa la regla de negocio y el punto del Controlador que lo invoca, dejando explícita la correspondencia entre la especificación y la implementación.

Mensaje	Método en Servidor (Modelo)	Manejador en ClienteHandler (Controlador)
IDENTIFY	Identificar(Usuario)	ManejarIdentify
STATUS	CambiarEstado(Usuario, Estado)	ManejarStatus
USERS	ObtenerUsuarios()	ManejarUsers
TEXT	EnviarTextoPrivado(...)	ManejarText
PUBLIC_TEXT	EnviarTextoPublico(...)	ManejarPublicText
NEW_ROOM	CrearSala(...)	ManejarNewRoom
INVITE	Invitar(...)	ManejarInvite
JOIN_ROOM	UnirseASala(...)	ManejarJoinRoom
ROOM_USERS	ObtenerUsuariosDeSala(...)	ManejarRoomUsers
ROOM_TEXT	EnviarTextoSala(...)	ManejarRoomText
LEAVE_ROOM	AbandonarSala(...)	ManejarLeaveRoom
DISCONNECT	Desconectar(Usuario)	(cierre de conexión en Atender)

Cuadro 2: Cada operación del protocolo tiene un único método responsable en el Modelo; el Controlador nunca decide reglas de negocio por su cuenta, corrigiendo así el problema de diseño de `conexionHandler` en la primera iteración.

4. Implementación

4.1 Tecnologías utilizadas

- **Lenguaje y runtime:** C# 12 sobre .NET 8 (LTS), tanto para el servidor como para el cliente.
- **Sockets:** `System.Net.Sockets.TcpListener` en el servidor para aceptar conexiones, y `TcpClient` / `NetworkStream` en ambos programas para leer y escribir el flujo de bytes.
- **Concurrencia:** `System.Threading.Thread` —un hilo por conexión aceptada en el servidor, y un hilo lector en segundo plano en el cliente— junto con la instrucción `lock` de C# (basada en `Monitor`) para proteger el estado compartido del servidor.
- **JSON:** `System.Text.Json`, incluida en el propio runtime de .NET; no se agregó ningún paquete NuGet externo al proyecto.
- **Organización del código:** tres proyectos de .NET (`chatComun`, `server`, `client`) reunidos en una sola solución (`ChatMulti.sln`).

4.2 Requisitos para ejecutar el proyecto

- SDK de **.NET 8** instalado (funciona igual en Linux, Windows o macOS).
- Para probar el chat entre dos máquinas distintas: ambas conectadas a la *misma* red local, con el puerto elegido para el servidor permitido en el firewall de la máquina que hace de servidor (por ejemplo, con `ufw allow <puerto>/tcp` en distribuciones basadas en Debian/Ubuntu).

4.3 Instrucciones de compilación y ejecución

```
# Desde la carpeta raíz del proyecto (donde está ChatMulti.sln):  
dotnet build ChatMulti.sln
```

```
# Terminal 1: servidor (el puerto es opcional; por defecto 5000)  
dotnet run --project server -- 5000
```

```
# Terminal 2, 3, ... : un cliente por cada usuario que se quiera conectar  
dotnet run --project client
```

Al arrancar, el cliente pregunta de forma interactiva el *host* (dirección IP del servidor, o `localhost` si ambos corren en la misma máquina), el puerto, y el nombre de usuario con el que se desea identificar. Una vez conectado, se escribe texto público directamente, o se usan comandos con prefijo `/` para el resto de las operaciones del protocolo (`/status`, `/msg`, `/newroom`, `/invite`, `/join`, `/roomusers`, `/roomtext`, `/leave`, `/quit`).

4.4 Estado de las pruebas

En la primera iteración (en C) sí se llegaron a escribir pruebas unitarias con la biblioteca `cmocka`, y funcionaron bien para ese propósito. Por restricciones de tiempo, esas pruebas **no** se portaron a la versión final en C#; queda como trabajo inmediato agregar un proyecto de pruebas (por ejemplo con `xUnit`) para `chatComun` y para la clase `Servidor`, que al no depender de sockets reales se puede probar de forma aislada.

5. Conclusiones

El mayor reto de todo el proyecto fue, sin duda, la programación concurrente —en particular durante la primera iteración en C, donde sincronizar hilos con mutex de bajo nivel y, al mismo tiempo, administrar memoria manualmente, resultó abrumador—. Pasar a C# no eliminó la necesidad de razonar cuidadosamente sobre concurrencia, pero sí quitó de en medio dos fuentes constantes de errores (la gestión manual de memoria y la simulación de objetos con *structs* y cabeceras), dejando espacio para concentrarse en el problema real: qué estado compartir y cómo protegerlo.

Escribir pruebas unitarias en C con *cmocka* fue, de hecho, una de las partes más sencillas y más valiosas del proceso: obligó a pensar en casos límite del protocolo desde antes de que aparecieran como bugs en producción, y es una práctica que se pretende retomar en C# como siguiente paso. De manera relacionada, el proyecto dejó claro que la primera versión de un programa rara vez es la definitiva, y que reconocer a tiempo cuándo conviene desechar un diseño (o una implementación completa, como ocurrió con la versión en C) y empezar de nuevo es tan importante como saber cuándo seguir insistiendo.

Esta fue también la primera vez que se usó *git* de forma seria para llevar el desarrollo; aunque nunca fue necesario recurrir a una versión anterior estable, tener ese respaldo disponible en todo momento dio mucha más tranquilidad para experimentar y refactorizar sin miedo a perder trabajo.

Posibles mejoras

- Agregar manejo de errores más claro en el cliente al fallar la conexión inicial (por ejemplo, distinguir un *timeout* de una conexión rechazada, en vez de una excepción genérica).
- Portar las pruebas unitarias de la versión en C a un proyecto de pruebas para la versión en C#.
- Si el protocolo creciera, migrar de objetos JSON dinámicos a clases (DTOs) fuertemente tipadas por cada tipo de mensaje.

Trabajo futuro / posible expansión

- **Autenticación:** permitir contraseñas o algún otro mecanismo para evitar que cualquiera se identifique con el nombre de otro usuario disponible.
- **Cifrado:** envolver la conexión TCP con TLS, para que los mensajes no viajen en texto plano dentro de la red local.
- **Privacidad de IPs:** evitar que quien tenga acceso al servidor pueda ver trivialmente la dirección IP de cada usuario conectado.
- **Escalabilidad:** para soportar mucho más allá de la decena de miles de usuarios concurrentes contemplados en el alcance, explorar múltiples instancias de servidor coordinadas por un sistema de mensajería (por ejemplo, un *message broker*), en vez de un único proceso con estado en memoria.
- **Interfaz gráfica:** gracias a haber seguido MVC, sustituir *VistaConsola* por una interfaz gráfica (WPF, Avalonia, o incluso una aplicación web) no debería requerir tocar el Modelo ni el Controlador del cliente.

6. Bibliografía

- Protocolo del chat multiusuario, documento proporcionado por el profesor (Canek) en el curso *Modelado y Programación* (material de curso, sin publicar).
- Microsoft. *System.Text.Json Namespace*. Documentación de la API de .NET. <https://learn.microsoft.com/en-us/dotnet/api/system.text.json>
- Microsoft. *TcpListener Class*. Documentación de la API de .NET. <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcplistener>
- Microsoft. *TcpClient Class*. Documentación de la API de .NET. <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcpclient>
- Microsoft. *Thread Class*. Documentación de la API de .NET. <https://learn.microsoft.com/en-us/dotnet/api/system.threading.thread>
- Microsoft. *The lock statement – ensure exclusive access to a shared resource*. Referencia del lenguaje C#. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/lock>
- cmocka. *cmocka – unit testing framework for C*. <https://cmocka.org/>